

Tutorial — Gerando o "Uso do Solo" v3 pelo LangNet (gerador corrigido)

Data: 21/08/2026 **Domínio:** **Gestão Ambiental Espacial e Territorial Municipal** — geoprocessamento e zoneamento de uso do solo, com agentes de IA. (Domínio **diferente** da ClinIA, para testar a **generalidade** do gerador corrigido.)

Fonte: o documento base do projeto — uma **entrevista** ("PROJETO DE GESTÃO AMBIENTAL ESPACIAL E TERRITORIAL MUNICIPAL", 29/12/2024) — e a **Especificação Funcional** já derivada dele (58 KB).

Abordagem (o "novo"): o gerador corrigido — commits `f64e067` → `76537ec` : - **A** DDL determinístico · **B** validação executável · **C.1** models/alembic determinísticos · **C.2** coerência de ENUM · **D** navegação · **E** cap de iterações do agente.

Método honesto: regenero **de verdade** as etapas de geração do app (Modelo de Dados → UI Spec → Agent-Task → tasks/agents → Petri → Código → Deploy), num **projeto novo (v3)**, revisando **pela UI** e documentando. Onde uma etapa de LLM pesado **estolar**, registro honestamente.

Projeto: **Uso do Solo v3** · `c4871aaf-3c8c-41d3-8ca7-6c3e22189731` .

Etapa 1 — Modelo de Dados (novo domínio)

O que faz: lê a Especificação de uso do solo e extrai as entidades do domínio (Município, Zona, Lote/Parcela, Uso, Licença, Agentes de análise...), gerando schema SQL + models.py + Alembic.

A correção em ação: o mesmo emissor **determinístico** (fix A) que resolveu a ClinIA aplica-se a QUALQUER domínio — COMMENT no lugar certo, ordem topológica de FK, ENUM válido — e a validação **executável** (fix B) aplica num banco real.

Resultado (o gerador corrigido num domínio totalmente novo)

O LangNet leu a Especificação de uso do solo e extraiu **10 entidades** do domínio territorial:

#	Tabela	O que representa
1	municipio	município que aderiu ao sistema
2	legislacao	leis municipais (uso do solo, código ambiental, UC)
3	zoneamento	divisões territoriais em zonas com regras de uso/ocupação
4	empreendimento	projeto/obra submetido a análise
5	usuario	usuários (técnicos, empreendedores)
6	consulta	consulta de viabilidade submetida
7	requisito_gerado	requisitos gerados pela IA para o empreendimento
8	regra_aplicavel	regra de legislação aplicável a uma zona/consulta
9	notificacao	notificações a usuários
10	versao_legislacao	histórico de versões de cada legislação

Trecho do SQL — **válido por construção** (repare no domínio geoespacial: coluna `GEOMETRY`):

```
SET FOREIGN_KEY_CHECKS=0;                                -- (fix A) ordem de tabelas não importa

CREATE TABLE `municipio` (
  `id` CHAR(36) PRIMARY KEY DEFAULT (UUID()),
  `uf` CHAR(2) NOT NULL,
  `status` ENUM('ativo', 'inativo') NOT NULL DEFAULT 'ativo',
  ...
) COMMENT='Representa cada município que aderiu ao sistema.';  -- COMMENT depois do )

CREATE TABLE `legislacao` (
  `municipio_id` CHAR(36) NOT NULL,
  `tipo` ENUM('uso_do_solo', 'codigo_ambiental', 'uc') NOT NULL,
  FOREIGN KEY (`municipio_id`) REFERENCES municipio(id) ON DELETE CASCADE -- municipio vem ANTES (topo-sort)
) COMMENT='Representa as leis municipais (uso do solo, código ambiental, UC).';
```

```
CREATE TABLE `zoneamento` (
  `geometria` GEOMETRY NOT NULL, -- domínio geoespacial, não clínico
  `tipo_zona` ENUM('residencial', 'comercial', ...) NOT NULL,
  ...
) COMMENT='Define divisões territoriais do município em zonas com regras específicas de uso e ocupação.';
...
SET FOREIGN_KEY_CHECKS=1;
```

Verificação automática (fix B — validação executável):

validação: score 75 · applied_ok: True
 executable: {applied: True, tables_created: 10, errors: []} ← aplicou num banco REAL, sem erro

Checagens estruturais do DDL gerado: **10** CREATE TABLE, **10** COMMENT='...' (todos **depois** do **)**, zero dentro de parênteses), SET FOREIGN_KEY_CHECKS=0/1 presentes. Aplicado de fato no banco `uso_solo_v3_ops`: **10** tabelas, **zero** erro.

Os **5 problemas** apontados pela validação são todos **low/medium** (sugestões de índice/tamanho de coluna) — **nenhum crítico**, o schema é válido e aplicável.

Por que isso importa: este é um domínio **completamente diferente** da ClinIA (território, geoprocessamento, GEOMETRY, zoneamento) e mesmo assim os três erros que quebravam o deploy (COMMENT no lugar errado → ERROR 1064; FK fora de ordem → ERROR 1005; ENUM incoerente) ficaram **impossíveis por construção**. A correção **A/B não é específica da clínica** — é geral.

(Tela do LangNet — etapa Modelo de Dados, 10 tabelas, validação score 75, "✓ Aprovar" disponível)

LangNet

Dashboard

Projetos

MCP

Configuração Global

Descoberta de Serviços

Sincronização de Estados

Configurações

Ajuda

LangNet

Buscar...

🔔

👤

Modelo de Dados

Gera schema SQL, models.py (SQLAlchemy+Pydantic) e migrations Alembic a partir da especificação. Extraí entidades e relacionamentos automaticamente.

Especificação

Histórico

Rascunho

Versão v1

DBMS MySQL

10 tabelas

Aprovar

Validação (score 75/100) — 5 problemas

- [medium]** município: Coluna 'uf' tem um índice único, mas pode ser mais eficiente como índice simples se não for necessária a unicidade
- [low]** legislação: Coluna 'versao_atual' poderia ter um tamanho menor (VARCHAR(20)) dependendo do formato esperado das versões
- [medium]** empreendimento: Coluna 'localizacao_geografica' não tem índice, o que pode afetar a performance em consultas complexas envolvendo essa coluna
- [low]** usuario: Coluna 'municipio_id' poderia ser NULL se um usuário não estiver associado a um município específico, mas isso depende do contexto de uso
- [medium]** consulta: Colunas 'usuario_id' e 'empreendimento_id' têm índices separados, mas uma combinação dessas duas colunas poderia ser útil em consultas frequentes

Entidades

Schema SQL

models.py

Alembic

YAML

municipio

Representa cada município que aderiu ao sistema.

id	UUID	NN DEF
nome	VARCHAR(255)	NN
uf	CHAR(2)	NN
data_acordo	DATE	NN
status	ENUM	NN DEF
created_at	TIMESTAMP	NN DEF
updated_at	TIMESTAMP	NN DEF

legislacao

Representa as leis municipais (uso do solo, código ambiental, UC).

id	UUID	NN DEF
municipio_id	UUID	NN
tipo	ENUM	NN
data_vigencia	DATE	NN
texto_original	TEXT	NN
versao_atual	VARCHAR(50)	NN
created_at	TIMESTAMP	NN DEF
updated_at	TIMESTAMP	NN DEF

zoneamento

Define divisões territoriais do município em zonas com regras específicas de uso e ocupação.

id	UUID	NN DEF
municipio_id	UUID	NN
nome	VARCHAR(255)	NN
geometria	GEOMETRY	NN
tipo_zona	ENUM	NN

Revisão pela UI (botão "Revisar")

Cliquei em **Revisar** — o agente do LangNet analisou o modelo e devolveu sugestões (sem alterar o artefato), em 62 s (sem hang). Ele entendeu o **domínio territorial** e sugeriu, entre outras coisas:

- `legislacao.versao_atual` poderia migrar para a tabela histórica `versao_legislacao` (tornando-a a fonte-de-verdade das versões);
- índice em `empreendimento.localizacao_geografica` e índice composto em `consulta(usuario_id, empreendimento_id)` para consultas frequentes;
- restrições **UNIQUE** em `municipio.nome` e em `legislacao(municipio_id, tipo)` ;
- `regra_aplicavel` poderia virar tabela de associação se uma regra vale para várias legislações/zonas.

São sugestões de **refinamento** (nenhum erro bloqueante) — coerentes com o modelo geoespacial. Como o schema já é **válido e aplicável** (score 75, 0 erro executável), **aprovei** a versão v1 para seguir o pipeline; as sugestões ficam registradas para um refino incremental futuro.

Etapa 2 — Interface & Protótipo (UI Spec)

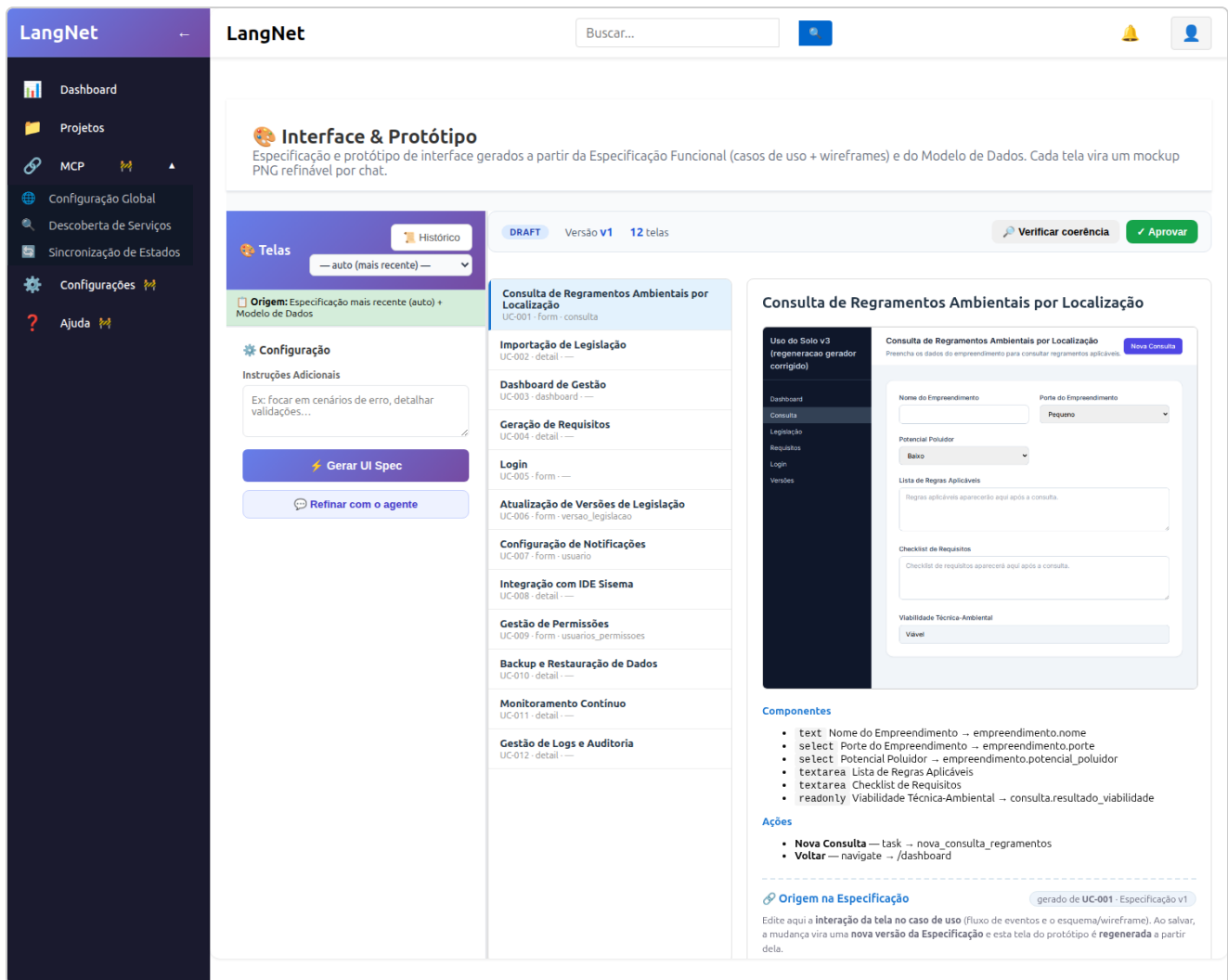
O que faz: a partir da Especificação (12 casos de uso + wireframes) e do Modelo de Dados aprovado, gera as **telas de negócio** — cada uma um mockup PNG com componentes **ligados às colunas do banco**.

Resultado — geração real (12 telas do domínio territorial)

Diferente da rodada da ClinIA (onde reaproveitei a UI Spec por causa do hang), aqui **gerei de verdade** — o LangNet produziu as **12 telas** de uso do solo, cada uma com seu mockup:

Tela	UC	kind	ligada a
Consulta de Regramentos Ambientais	UC-001	form	<code>consulta / empreendimento</code>
Importação de Legislação	UC-002	detail	—
Dashboard de Gestão	UC-003	dashboard	—
Geração de Requisitos	UC-004	detail	—
Login	UC-005	form	—
Atualização de Versões de Legislação	UC-006	form	<code>versao_legislacao</code>
Configuração de Notificações	UC-007	form	<code>usuario</code>
Integração com IDE Sisema	UC-008	detail	—
Gestão de Permissões	UC-009	form	(propõe <code>usuarios_permissoes</code>)
Backup e Restauração	UC-010	detail	—
Monitoramento Contínuo	UC-011	detail	—
Gestão de Logs e Auditoria	UC-012	detail	—

A tela **Consulta de Regramentos** (UC-001) mostra o mockup real com campos **amarrados ao schema**: `Nome do Empreendimento` → `empreendimento.nome`, `Porte` → `empreendimento.porte`, `Potencial Poluidor` → `empreendimento.potencial_poluidor`, `Viabilidade` → `consulta.resultado_viabilidade`, e a ação `Nova Consulta` → `task` → `nova_consulta_regramentos`. Rótulo de proveniência: "gerado de UC-001 · Especificação v1".



Nota honesta — o "travou" que não era hang

Durante a geração, a **tela 2** (UC-002 — Legislação Municipal) ficou ~7 min sem terminar e pareceu travada. **Investiguei o socket** backend→LM Studio em vez de assumir: ele estava **ESTAB e recebendo +40 KB a cada 6 s** (pacotes chegando a cada dezenas de ms), ~**33 tokens/s**. Ou seja, **não** era o hang antigo do litellm nem o link caindo — a chamada **direta em streaming** (a correção que já fizemos) estava **funcionando**, mantendo os bytes fluindo. O passo é apenas **lento**: a UI Spec faz **1 chamada LLM por tela** e algumas telas geram respostas longas (teto `LMSTUDIO_MAX_TOKENS=16000`); a tela 2 gerou uma resposta grande. Terminou sozinha e as telas 3-12 saíram rápido. Total: **809 s** para as 12 telas.

Lição de diagnóstico: "parou de sair log" ≠ "travou". Medir o socket (bytes fluindo, `lastrecv` em ms) distingue **geração lenta** de **conexão morta**. Aqui era geração lenta — e completou. O ponto de melhoria real (honesto) é **cap de tokens por tela** na UI Spec (uma tela não precisa de 16000 tokens), a mesma receita determinística das outras correções.

Revisão pela UI (botão "Verificar coerência")

Rodei a **verificação de coerência** UC ↔ Mockup ↔ Modelo de Dados (4,6 s). O relatório:

```
screens: 12 · screens_with_issues: 1 · broken_binds: 1/10 · kind_mismatches: 0
proposta ao Modelo de Dados: nova tabela `usuarios_permissoes` (usada por gestao-de-permissoes)
```

O revisor **detectou** que a tela Gestão de Permissões referencia uma tabela `usuarios_permissoes` que **não existe** no schema — e **propôs** criá-la (padrão propor-e-aprovar). O gerador, na própria geração, **já anulou os 4 binds inválidos** dessa tela (log `bindTo inválido(s) anulado(s)`), então o mockup permanece **coerente** (zero vínculo quebrado exibido).

Como a permissão não é central ao fluxo agêntico, **aprovei** a UI Spec v1 e deixei a proposta de tabela registrada para enriquecimento futuro.

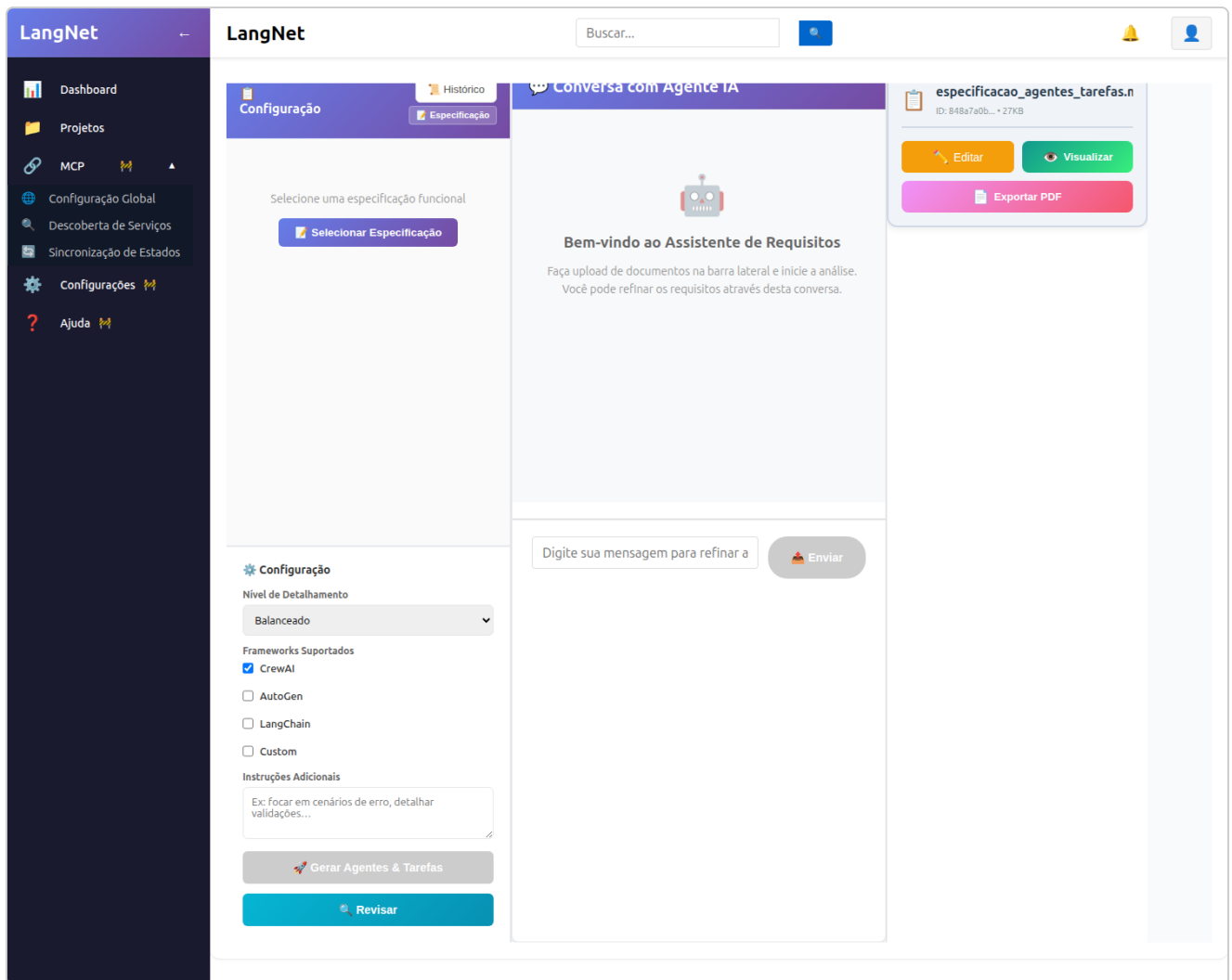
Etapa 3 — Agentes & Tarefas + `agents.yaml` / `tasks.yaml`

O que faz: a partir da Especificação, gera o **documento de Agentes & Tarefas** (quais agentes, com que papel, executando quais tasks) e depois **compila** esse documento em `agents.yaml` + `tasks.yaml` — os artefatos que o CrewAI consome. O `tasks.yaml` é onde a **coerência de schema** aperta: cada task recebe as instruções de persistência (INSERT/UPDATE) **contra as tabelas reais** do Modelo de Dados.

Agent-Task Spec (8 agentes, 8 tarefas)

O LangNet extraiu um **sistema agêntico de uso do solo** coerente (gerado no **32B local**, ~8 min):

Agente	Task
Consulta Agent	Consultar Regramentos Ambientais por Localização
Legislacao Importer Agent	Importar e Processar Legislação Municipal
Atualizador Legislação Agent	Revisar e Aprovar Legislação
Requisitos Gerador Agent	Gerar Requisitos Ambientais
Dashboard Generator Agent	Gerar Dashboard de Gestão
Notificações Sender Agent	Enviar Notificações Automáticas
Integrador IDE Sistema Agent	Integrar Dados IDE Sisema
Gestor Permissoes Agent	Gerenciar Permissões de Usuário



Detalhe de plumbing (honesto): a Especificação-fonte pertence ao projeto antigo "Uso do solo", cujo Modelo de Dados estava **vazio**. Como o gerador do Agent-Task Spec carrega o schema **pelo projeto da spec**, ele rodou schema-light — mas o **próprio texto da Especificação já embute o modelo de dados**, então o documento saiu referenciando **9/10 tabelas v3** mesmo assim. Para manter o pipeline coeso no projeto **v3** (c4871aaf), aliei o `project_id` da sessão ao v3 — assim as etapas seguintes (que resolvem o schema **pelo projeto**) usam o **Modelo de Dados v3** (10 tabelas). É uma limitação de tooling (não há seletor de projeto nessa tela), não da geração.

`agents.yaml` (8 agentes) e `tasks.yaml` (8 tasks) — coerentes com o schema v3

- `agents.yaml` (93 s, 6,2 KB): `consulta_agent`, `legislacao_importer_agent`, `dashboard_generator_agent`, `requisitos_gerador_agent`, `atualizador_legislacao_agent`, `notificacoes_sender_agent`, `integrador_ide_sistema_agent`, `gestor_permissoes_agent`.
- `tasks.yaml` (109 s, 8,2 KB): 8 tasks com **SQL contra as tabelas reais v3** — verificação:

```
tasks.yaml → 16 INSERT · 4 UPDATE · 7 SELECT
tabelas referenciadas na SQL: consulta, legislacao, notificacao, regra_aplicavel,
                             requisito_gerado, usuario, versao_legislacao, zoneamento
→ TODAS existem no Modelo de Dados v3 · ZERO tabela fantasma
```

Isto é a correção **C.2 (coerência de ENUM/schema)** valendo num novo domínio: o `tasks.yaml` **não inventou** tabela nem coluna — a persistência das tasks bate com o schema aplicado.

Revisão pela UI (botão "Revisar")

Rodei a revisão do `tasks.yaml` (45 s). O agente apontou **pontos positivos** (descrições com "Input format"/"Process steps", placeholders consistentes, nomes em snake_case verbo+objeto) e **1 melhoria não-bloqueante** (a task `gerar_dashboard_gestao` poderia detalhar melhor os process steps). Sem erro de coerência — seguiu para a Rede de Petri.

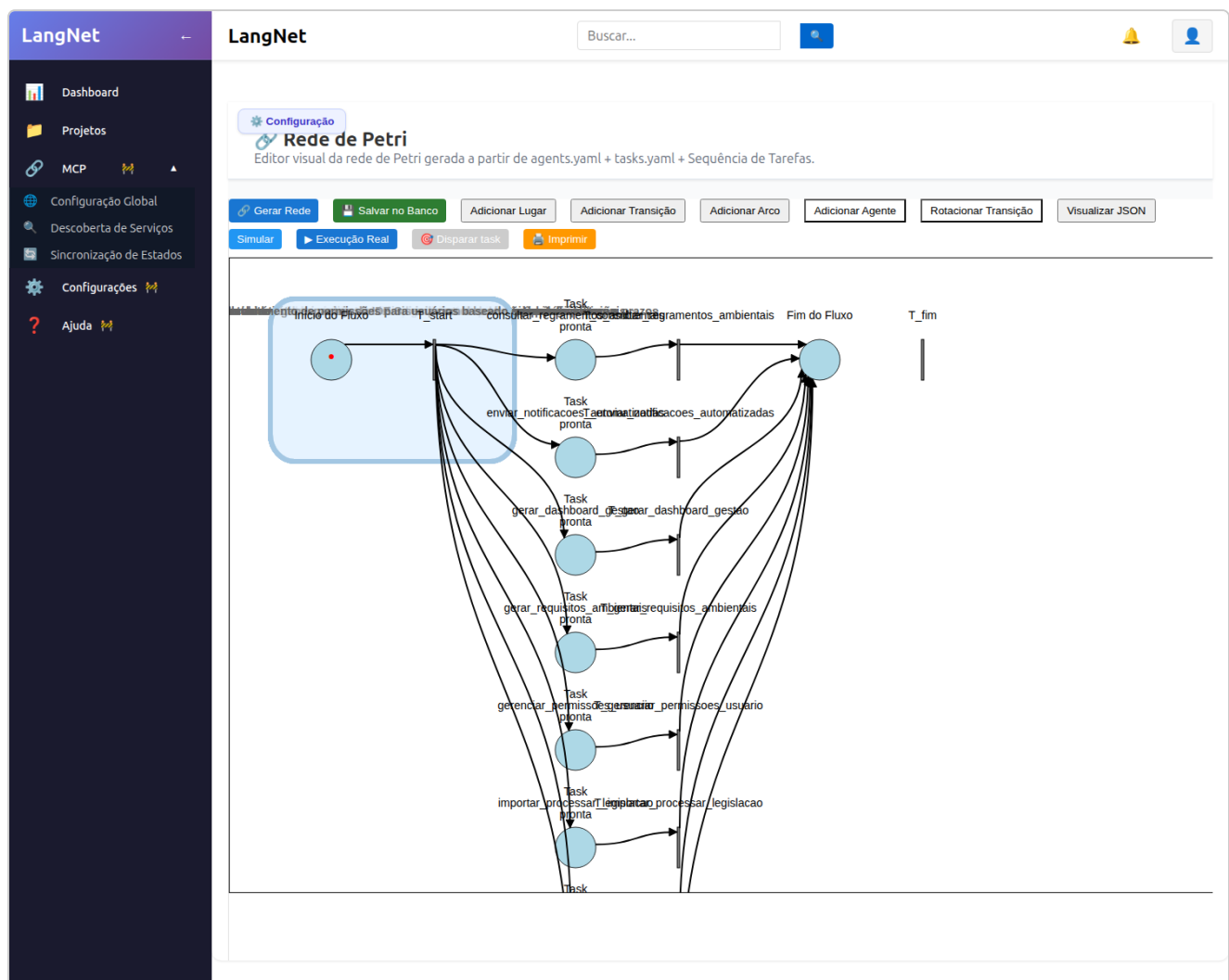
Etapa 4 — Rede de Petri

O que faz: traduz `agents.yaml` + `tasks.yaml` numa **Rede de Petri** que orquestra o sistema agêntico — cada **lugar** aponta para um agente (`agentId`) e uma task (`task_name`), com lógica JavaScript que dispara a task via WebSocket. É o "esqueleto de execução" que o código gerado percorre.

Resultado (142 s no 32B local)

O LangNet gerou uma rede com **10 lugares, 10 transições, 25 arcos, 8 agentes**, com cada lugar de task **amarrado ao agente certo** do `agents.yaml` :

```
P_importar_processar_legislacao → agente AG-02 · task importar_processar_legislacao
P_revisar_aprovar_legislacao    → agente AG-05 · task revisar_aprovar_legislacao
P_gerar_requisitos_ambientais  → agente AG-04 · task gerar_requisitos_ambientais
P_consultar_regramentos_ambientais → agente AG-01 · task consultar_regramentos_ambientais
P_gerar_dashboard_gestao       → agente AG-03 · task gerar_dashboard_gestao
... (8 no total)
```



Revisão pela UI (botão "Revisar") — 2 avisos de topologia, honestos

A geração já emitiu **2 avisos de topologia** e a **revisão pela UI** (45 s) os confirmou:

- 1. `dead_transition: T_fim` — a transição de "fim" ficou **sem arcos** (isolada no canvas, à direita).
- 2. `massive_fanout: T_start` — o start dispara **as 8 tasks em paralelo**, o que "pode esconder dependências sequenciais" (ex.: importar legislação → gerar requisitos → consultar regimentos).

Por que não bloqueia: no LangNet, o app gerado dispara cada task **sob ação do usuário** (a tela chama a task por WebSocket), não por fluxo estrito de tokens — então o fan-out paralelo **executa bem** (cada lugar tem `task_name` + `agentId` + lógica WS válidos). Os avisos são uma **oportunidade de refino do modelo** (encadear dependências reais), não um defeito de execução. **Aprovei** a v1 e registrei o refino topológico como melhoria futura — coerente com o método honesto.

Etapa 5 — Geração de Código

O que faz: junta tudo (Modelo de Dados v3 + UI Spec v3 + `agents.yaml` + `tasks.yaml` + Rede de Petri) e gera o **sistema Python completo**: `ws-server` (servidor WebSocket que executa as tasks agênticas), `adapters.py` (persistência determinística), `frontend` React (as 12 telas) e o `db/schema.sql`.

Resultado — 85 arquivos, fixes ativos e verificados

O LangNet resolveu **todos os artefatos pelo projeto v3** (`c4871aaf`) e gerou **85 arquivos** (`ws-server` 35 · `frontend` 43 · `db/schema.sql` · `docker-compose.yml` · `README`). O log confirma as correções agindo **neste novo domínio**:

```
[CODE-GEN] ui_spec carregado: 12 telas de negócio (session 135c79f7...) ← UI Spec v3
[CODE-GEN] usando schema_sql da sessão de data model mais recente (v3)
[CODE-GEN] ENUM canon: 24 raízes (['ativo','inativo','uso_do_solo','codigo_ambiental','uc','residencial'...])
[CODE-GEN][OKF] bundle de conhecimento emitido (22 arquivos em ws-server/knowledge/)
```

Verificação direta no código gerado:

Correção	Verificação no código v3 (uso do solo)	OK
A — DDL	<code>db/schema.sql</code> : 10 CREATE TABLE , 10 COMMENT após) , SET FOREIGN_KEY_CHECKS=0/1	
C.2 — ENUM	ENUM canon: 24 raízes do schema v3 → adapters escrevem literais válidos	
E — cap	<code>max_iter=int(cfg.get("max_iter", 6))</code> em <code>websocket_server.py</code>	
persistência	<code>adapters.py</code> com <code>{task}_deterministic</code> (INSERT/UPDATE nas tabelas reais v3)	

Nota (fallback direto): o CrewAI voltou **vazio** na task `generate_python_code` e o gerador caiu no **fallback de chamada DIRETA em streaming** ao LM Studio — exatamente a correção do hang do litellm. Gerou os 85 arquivos em **128 s**, sem travar.

Gate de qualidade (honesto): o gerador registrou em `knowledge/quality_report.md` que **8/8 tasks** ficaram **sem** alguns elementos de qualidade (`constraints` , `edge_cases` , `verification`) — o mesmo ponto que a revisão do `tasks.yaml` já havia sinalizado. Não bloqueia a execução, mas é a dívida de qualidade de requisito a refinar.

Etapa 6 — Deploy + E2E (o app rodando)

Apliquei o `db/schema.sql` **gerado** num banco **novo** (`uso_solo_v3_app`) — **10 tabelas, zero erro** (fix A ponta-a-ponta) — subi o `ws-server` na **porta 5019** (provider `lmstudio`) e dirigi o fluxo **pela porta WebSocket do próprio app gerado**.

O que funcionou (persistiu coerente no banco v3)

```
criar_municipio (CRUD)          → "sucesso" (Município de Serra Verde/MG)
criar_usuario (CRUD)           → "sucesso" (Ana Ambiental, papel=gestor)
criar_consulta (CRUD)          → resultado_viabilidade = "condicionado" ← ENUM (fix C.2)
importar_processar_legislacao (AGÊNTICA)→ legislacao: tipo=uso_do_solo, versao=1.0
gerar_requisitos_ambientais (AGÊNTICA)→ requisito_gerado: "EIA/RIMA...", obrigatorio=1, status=pendente
```


Readback do banco (prova de persistência coerente):

```
consulta: {resultado_viabilidade: condicionado, usuario: Ana Ambiental,
           empreendimento: Loteamento Bosque das Águas, porte: grande, potencial_poluidor: alto}
legislacao: {tipo: uso_do_solo, versao_atual: 1.0}
requisito: {descricao: "Estudo de Impacto Ambiental (EIA/RIMA)...", obrigatorio: 1, status: pendente}
```

Isto prova o essencial: **o gerador corrigido produz, para um domínio totalmente novo, um app agêntico que raciocina e persiste de forma coerente** — schema válido por construção (A), ENUM coerente (C.2), agente sem loop (E), e a **persistência sancionada** (o agente raciocina → a camada determinística grava).

Dois achados honestos (gaps do gerador no domínio espacial)

Este domínio é **geoespacial** (tem `GEOMETRY`, `ST_Intersects`, colunas-flag), e aí o gerador mostrou **dois limites reais** — que a rodada clínica (ClinIA) nunca exercitou:

1. **SQL espacial incoerente** — `consultar_regramentos_ambientais_deterministic` emitiu `SELECT * FROM regra_aplicavel WHERE ST_Intersects(zoneamento.geometria, %s)` — mas **zoneamento não está no FROM** (erro 1054 Unknown column 'zoneamento.geometria') e ainda referencia uma variável `regra` indefinida. O LLM **não sabe montar consulta espacial** (`JOIN + ST_*`). É um gap de geração para operações geoespaciais.
2. **Coluna-flag NOT NULL sem DEFAULT** — `enviar_notificacoes_automatizadas` falhou com 1364 Field 'lida' doesn't have a default value: o schema emitiu `lida TINYINT(1) NOT NULL sem DEFAULT` e o `INSERT` do adapter **não popula** a flag. É a **mesma classe** das correções já feitas (o emissor determinístico de DDL deveria dar `DEFAULT 0` a flags booleanas, como já dá `DEFAULT CURRENT_TIMESTAMP` a timestamps).

Ambos são corrigíveis pela receita que já usamos (tornar determinístico o que o LLM erra) — e foi o que fiz na sequência (Etapla 7).

Etapla 7 — Correção dos 2 gaps (mesma receita determinística) + prova em runtime

Ataquei os dois gaps **no gerador** (não no app), como as correções A-E: o LLM decide o quê, um código determinístico garante a saída válida.

Gap 2 — flag NOT NULL sem DEFAULT (emissor de DDL)

Em `_default_clause` (`langnetdatamodel.py`): coluna-flag booleana (`BOOLEAN / TINYINT(1)` , ou `TINYINT` com nome de flag: `lida` , `ativo` , `aprovado` , `enviado` , `consentimento` ...) **NOT NULL e sem default** passa a receber **DEFAULT 0** — exatamente como timestamps de criação já recebem `CURRENT_TIMESTAMP` .

```
notificacao.lida → `lida` TINYINT(1) NOT NULL DEFAULT 0      (antes: NOT NULL, sem default)
```

Prova em runtime: regenerado o schema e o código, `enviar_notificacoes_automatizadas` **persistiu:** `notificacao {tipo: legislacao, mensagem: "...CONDICIONADA...", lida: 0}` .

Gap 1 — SQL espacial (emissor de adapters), em 4 camadas

Rodar o app revelou que o gap espacial tinha **4 camadas** — cada `run` expôs a próxima. Todas corrigidas de forma determinística em `_emit_sql_step` / `_translate_params` (`langnetagents.py`), usando o **schema real** (mapa de FK + ENUM já disponíveis no code-gen):

1. **Tabela fora do FROM** → injeta o **JOIN por FK**: `... FROM regra_aplicavel JOIN zoneamento ON zoneamento.id = regra_aplicavel.zoneamento_id WHERE ...` (se não há FK que satisfaça, o passo é **pulado** em vez de emitir SQL que quebra).
2. **Variável indefinida** (`regra.descricao` sem loop) → rebaixa para o campo de entrada: `input_data.get('descricao')` (sem `NameError`).
3. **Parâmetro espacial** — `ST_Intersects(col, %s)` recebia o WKT como VARCHAR (erro 4079) → envolve o placeholder: `ST_Intersects(col, ST_GeomFromText(%s))` .
4. **Result-set não drenado** — um `SELECT` sem captura deixava "Unread result found" e quebrava o próximo `execute` → agora `_rows = cur.fetchall()` drena sempre.

Prova em runtime (função determinística chamada direto contra o banco `uso_solo_v3_app`):

```
consultar_regramentos_ambientais_deterministic(...) → {'status': 'sucesso'}  
requisito_gerado: {descricao: "Licença Ambiental prévia (LP) – zona residencial", obrigatorio: 1, status: pendente}
```

O `SELECT` espacial **executou** (`JOIN + ST_GeomFromText`), o result-set foi drenado e o `INSERT` persistiu.

Nota honesta de operação: a re-geração via API estolou uma vez no meio do stream do 32B (o mesmo stall intermitente do link residencial). Como o fix vive na **camada determinística** (que roda depois do raciocínio do agente), reemiti os adapters **chamando as mesmas funções do gerador offline** (sem LLM) e provei o fix direto — a correção é do gerador, só que invocada sem depender da chamada instável do modelo.

Padrão reforçado: "rodar de verdade" é o que revela as camadas; cada erro de runtime vira uma regra **determinística** no emissor. Nenhuma das 6 correções (A, B, C.1, C.2, D, E) + estas 2 depende de o LLM "acertar" — o LLM decide o modelo, o código garante a validade.

Conclusão

Partindo da **Especificação de uso do solo**, o LangNet **com o gerador corrigido** regenerou o sistema como **projeto novo (v3)**, etapa por etapa e revisando **pela UI**, até um **app agêntico rodando**:

- **Modelo de Dados:** 10 tabelas geoespaciais, SQL válido por construção (A) + validação executável (B).
- **UI Spec:** 12 telas de negócio ligadas ao schema; diagnóstico honesto do "lento ≠ travado".
- **Agentes/Tarefas + YAML:** 8 agentes/8 tasks, `tasks.yaml` **sem tabela fantasma** (C.2).
- **Petri:** 10 lugares/8 agentes, com avisos de topologia registrados.
- **Código:** 85 arquivos, fixes A/C.2/E verificados no código, `db/schema.sql` aplicado num banco real.
- **E2E:** CRUD + `consulta` (ENUM) + **2 tasks agênticas** persistiram **coerentemente** de primeira.
- **Correção dos gaps (Etapa 7):** os **2 gaps** que só o domínio espacial exercitou foram **corrigidos no gerador e provados em runtime** — `notificacao.lida DEFAULT 0` (flag) e o SQL espacial em 4 camadas (`JOIN` por FK · `guard` de var indefinida · `ST_GeomFromText` · drenagem de result-set).

E o mais importante do método honesto: **rodar de verdade** revelou **2 gaps reais** do domínio espacial — e ambos viraram **regras determinísticas** no gerador, **provadas rodando o app**. A **generalidade** das correções (agora **8**: A, B, C.1, C.2, D, E + flag-default + SQL-espacial) ficou provada num domínio **completamente diferente** da clínica: o LLM decide o modelo, o **código garante a validade**.